

RPG SKILL CHECKER & COMPANION

Especificação de Casos de Uso

Equipe 9 | Versão 1.0 | Mar/2026

Histórico de Revisão

Data	Versão	Descrição	Autor
Mar/2026	1.0	Versão inicial consolidada do Documento de Casos de Uso.	Equipe 9

Conteúdo

1 introdução

2 Objetivo e Escopo

3 Atores

3.1 Jogador

3.2 Mestre (GM)

3.3 Sistema

4 Requisitos Não-Funcionais Relacionados

5 Arquitetura e Sistemas Relacionados

5.1 Arquitetura da Aplicação

5.2 Tecnologias Utilizadas

6 Premissas e Restrições

7 Especificação de Casos de Uso

7.1 Cadastrar e Autenticar Usuário

7.1.1 Descrição

7.1.2 Pré-Condições

7.1.3 Fluxo de Eventos

7.1.3.1 Fluxo Básico

7.1.3.2 Subfluxos

7.1.3.3 Fluxos Alternativos

7.1.3.4 Exceções

7.1.4 Pós-Condições

7.1.5 Requisitos Especiais

7.1.6 Regras de Negócio

7.1.7 Interfaces com Usuário

7.1.8 Demais Interfaces

7.1.8.1 Interfaces de Software

7.1.8.2 Interfaces de Hardware

7.1.9 Diagrama de Caso de Uso

7.1.10 Observações

7.2 Criar e Gerenciar Ficha de Personagem D&D 5e

7.2.1 Descrição

7.2.2 Pré-Condições

- 7.2.3 Fluxo de Eventos
 - 7.2.3.1 Fluxo Basico
 - 7.2.3.2 Subfluxos
 - 7.2.3.3 Fluxos Alternativos
 - 7.2.3.4 Exceções
- 7.2.4 Pós-Condições
- 7.2.5 Requisitos Especiais
- 7.2.6 Regras de Negócio
- 7.2.7 Interfaces com Usuário
- 7.2.8 Demais Interfaces
 - 7.2.8.1 Interfaces de Software
 - 7.2.8.2 Interfaces de Hardware
- 7.2.9 Diagrama de Caso de Uso
- 7.2.10 Observações
- 7.3 Rolar Dados Virtuais
 - 7.3.1 Descrição
 - 7.3.2 Pré-Condições
 - 7.3.3 Fluxo de Eventos
 - 7.3.3.1 Fluxo Basico
 - 7.3.3.2 Subfluxos
 - 7.3.3.3 Fluxos Alternativos
 - 7.3.3.4 Exceções
 - 7.3.4 Pós-Condições
 - 7.3.5 Requisitos Especiais
 - 7.3.6 Regras de Negócio
 - 7.3.7 Interfaces com Usuário
 - 7.3.8 Demais Interfaces
 - 7.3.8.1 Interfaces de Software
 - 7.3.8.2 Interfaces de Hardware
 - 7.3.9 Diagrama de Caso de Uso
 - 7.3.10 Observações

8 Rastreabilidade entre os Requisitos

9 Não Requisitos (Fora do Escopo da Fase 1)

10 Riscos Identificados

1. Introdução

Este documento consolida as especificações de Casos de Uso do RPG Skill Checker & Companion, sistema desenvolvido pela Equipe 9 no âmbito da disciplina de Análise e Projeto de Software. O material foi compilado a partir do Documento de Requisitos de Software e das especificações individuais dos casos de uso UC-01, UC-02 e UC-03.

O RPG Skill Checker & Companion é uma plataforma digital multiplataforma (web, iOS e Android) voltada a jogadores e mestres de RPG de mesa. O sistema visa eliminar a dependência de fichas físicas, calculadoras manuais e ferramentas estrangeiras de custo elevado, oferecendo uma solução unificada em português com suporte a D&D 5e (Fase 1) e futuramente ao sistema Ordem Paranormal (Fase 3).

O documento descreve a interação entre usuários e sistema e serve de base para o desenvolvimento, testes e validação do projeto.

2. Objetivo e Escopo

O diagrama abaixo representa os casos de uso gerais do sistema:

Ator	Casos de Uso Relacionados
Jogador	UC-01 Cadastrar/Autenticar; UC-02 Criar/Editar Ficha; UC-03 Rolar Dados
Mestre (GM)	UC-01 Autenticar; UC-03 Rolar Dados; UC-04 Gerenciar Sessão (Fase 2)
Sistema	Calcular modificadores; Persistir dados; Transmitir via WebSocket

3. Atores

3.1 Jogador

Usuário autenticado que pode cadastrar-se, criar e gerenciar fichas de personagens D&D 5e e realizar rolagens de dados. Acessa o sistema via navegador web (Chrome 110+, Firefox 110+, Safari 16+) ou aplicativo mobile (iOS 14+ / Android 10+).

3.2 Mestre (GM)

Usuário autenticado com perfil de mestre de jogo. Pode autenticar-se, realizar rolagens de dados e gerenciar sessões (Fase 2). Possui as mesmas permissões básicas que o Jogador para UC-01 e UC-03.

3.3 Sistema

Ator interno responsável por calcular modificadores de atributos e bônus de proficiência, persistir dados no banco de dados PostgreSQL 16, transmitir resultados de rolagens via WebSocket em sessões ativas e validar tokens JWT nas requisições.

4. Requisitos Não-Funcionais Relacionados

ID	Categoria	Descrição
RNF-01	Desempenho	O tempo de resposta da API deve ser inferior a 300ms no percentil 95 (P95) com até 500 usuários simultâneos. Seção 6 – Doc. de Visão Equipe 9.
RNF-02	Disponibilidade	O sistema deve ter uptime mínimo de 99,5% mensal. Hospedagem em VPS/Cloud (AWS EC2 ou Railway) – Seção 4.2 Doc. de Visão.
RNF-03	Segurança	Autenticação via JWT (access token 15min + refresh token 7 dias). Senhas com BCrypt fator 12. HTTPS obrigatório. Rate limiting: 60 req/min por IP – Seção 6 Doc. de Visão Equipe 9.
RNF-04	Portabilidade	Interface responsiva para web (Chrome 110+, Firefox 110+, Safari 16+) e mobile (iOS 14+ / Android 10+ via React Native) – Seção 7.2 Doc. de Visão.
RNF-05	Escalabilidade	Arquitetura modular em containers Docker para permitir escalonamento horizontal e adição de novos sistemas de RPG sem refatoração total – Seção 7.2.
RNF-06	Manutenibilidade	Cobertura de testes unitários $\geq 80\%$ no backend (JaCoCo). API documentada via Swagger/OpenAPI 3.0 – Seção 6 Doc. de Visão Equipe 9.

5. Arquitetura e Sistemas Relacionados

5.1 Arquitetura da Aplicação

A arquitetura segue o padrão de camadas com separação clara de responsabilidades, conforme definido na Seção 8 do Documento de Visão – Equipe 9:

Camada	Descrição
Apresentação Web	React 18 + TypeScript (SPA). Consome API REST e WebSocket via Axios e STOMP.js.
Apresentação Mobile	React Native + Expo. Mesma lógica do frontend web, adaptada para iOS e Android.
API Gateway / BFF	Spring Boot 3 (Java 21). Endpoints REST e WebSocket. Autenticação e roteamento.
Serviço de Regras	Engine D&D 5e em Java puro: modificadores, CA, HP, salvaguardas, proficiências.
Serviço de Sessões	WebSocket (STOMP + SockJS). Estado das salas mantido no Redis 7.
Persistência	PostgreSQL 16 (dados relacionais) + Redis 7 (cache e estado de salas ao vivo).
Segurança	Spring Security + JWT. HTTPS obrigatório. BCrypt fator 12. Rate limiting por IP.

5.2 Tecnologias Utilizadas

Tecnologia	Categoria	Utilização	Disponibilidade
Java 21 (LTS)	Backend	Spring Boot 3 – linguagem principal do servidor.	https://adoptium.net/
Spring Boot 3	Backend	Framework REST, WebSocket, Security, JPA.	https://spring.io/
React 18 + TypeScript	Frontend Web	SPA com Vite. Consome API e WebSocket.	https://react.dev/
React Native + Expo	Mobile	App iOS e Android a partir de código compartilhado.	https://expo.dev/
PostgreSQL 16	Banco de Dados	Persistência relacional. Suporte a JSONB.	https://www.postgresql.org/
Redis 7	Cache / Sessões	Estado de salas ao vivo. Cache do compêndio.	https://redis.io/

Docker + Compose	Infra	Containerização de todos os serviços.	https://www.docker.com/
GitHub Actions	CI/CD	Pipeline de build, testes e deploy automatizados.	https://github.com/features/actions
JWT (JJWT)	Segurança	Geração e validação de access/refresh tokens.	https://github.com/jwt/jwt

6. Premissas e Restrições

ID	Descrição
P-01	O conteúdo SRD do D&D 5e será utilizado sob licença Creative Commons, sem custo (Seção 1.4.1 e 4.3 do Doc. de Visão – Equipe 9).
P-02	O sistema Ordem Paranormal será implementado somente a partir da Fase 3, após aprovação formal da equipe (Seção 4.3 do Doc. de Visão).
P-03	Toda mudança de escopo nas HUs deve ser aprovada pela Equipe 9 antes da execução, com análise de impacto no roadmap.
P-04	O ambiente de testes (Docker local ou Railway) deve estar disponível no mínimo 2 dias úteis antes do início de cada sprint de testes.
P-05	O aplicativo mobile não deve exceder 80 MB de tamanho de download (Seção 7.2 – Restrições Aplicáveis do Doc. de Visão).
P-06	Nenhum dado de usuário pode ser armazenado sem criptografia em repouso (Seção 7.2 – Restrições Aplicáveis do Doc. de Visão).
P-07	A arquitetura deve ser modular para permitir adição de novos sistemas de RPG sem refatoração total (Seção 7.2 do Doc. de Visão).

7. Especificação dos Casos de Uso

UC-01 – Cadastrar e Autenticar Usuário

ID do Caso de Uso: UC-01

Nome: Cadastrar e Autenticar Usuário

Atores: Jogador, Mestre (GM) – usuário não autenticado

Prioridade: Crítico – Prioridade 1, Fase 1 MVP (Seção 5 Doc. de Visão Equipe 9)

Referência: HU-01; RF0001; Seção 3.2 (Necessidade 3); Seção 9.2 (tabela users)

1. Descrição

Este caso de uso permite que um novo usuário se cadastre no sistema informando e-mail e senha, ou que um usuário já cadastrado realize login para acessar suas fichas e sessões. O sistema valida as credenciais, armazena a senha como hash BCrypt (fator 12) no campo password_hash (VARCHAR 255) da tabela users e retorna tokens JWT para autenticação nas demais funcionalidades do RPG Skill Checker & Companion.

2. Pré-condições

- O sistema está disponível e o banco de dados PostgreSQL 16 está acessível.
- O usuário possui acesso à internet e um e-mail válido.
- Para login: o usuário já possui cadastro na tabela users do sistema.

3. Fluxo de Eventos

3.1 Fluxo Básico

1. O usuário acessa a tela de cadastro (/cadastro) ou login (/login).
2. Para cadastro: o usuário preenche os campos E-mail, Senha e Confirmar Senha e clica em 'Criar Conta'.
3. O sistema valida o formato do e-mail (RFC 5322) e verifica unicidade na tabela users (campo email UNIQUE).
4. O sistema valida a senha: mínimo 8 caracteres, ao menos uma letra maiúscula e um número [RN-01].
5. O sistema gera o hash BCrypt (fator 12) da senha e insere o registro na tabela users com id (UUID), email, password_hash, created_at e updated_at.

6. O sistema envia e-mail de confirmação e gera access token JWT (15min) e refresh token (7 dias) [RN-02].
7. O sistema redireciona o usuário autenticado para o painel /dashboard.
8. Para login: o usuário preenche E-mail e Senha e clica em 'Entrar'.
9. O sistema verifica as credenciais contra o hash BCrypt armazenado na tabela users.
10. O sistema gera access token JWT (15min) e refresh token (7 dias) [RN-02].
11. Se 'Manter conectado' estiver marcado, o refresh token é armazenado em cookie HttpOnly com validade de 7 dias.
12. O sistema redireciona o usuário para o painel /dashboard.

3.2 Subfluxos

(S01) – Renovação de Token (Refresh)

13. O access token JWT expira (após 15 minutos).
14. O cliente envia o refresh token ao endpoint POST /auth/refresh.
15. O sistema valida o refresh token no Redis 7.
16. O sistema gera novo access token JWT (15min) e retorna ao cliente.
17. O fluxo principal é retomado com o novo access token.

3.3 Fluxos Alternativos

(A01) – E-mail já cadastrado

Pré-condições: O e-mail informado já existe na tabela users (violação de constraint UNIQUE).

18. O sistema detecta e-mail duplicado na tabela users.
19. O sistema exibe MSG001: 'E-mail já cadastrado. Utilize outro e-mail ou faça login.'
20. O campo E-mail recebe destaque visual de erro (borda vermelha).
21. Nenhum registro é inserido na tabela users.

Retorno: O usuário permanece na tela de cadastro para corrigir o e-mail.

(A02) – Usuário marca 'Manter conectado'

Pré-condições: O checkbox 'Manter conectado' está marcado no formulário de login.

22. Após login bem-sucedido, o sistema armazena o refresh token em cookie HttpOnly.
23. O refresh token tem validade de 7 dias no Redis 7.
24. O acesso é mantido sem necessidade de novo login por até 7 dias.

Retorno: O fluxo básico continua a partir do passo 12 (redirecionamento ao /dashboard).

3.4 Exceções

(E01) – Credenciais inválidas no login

Pré-condições: A senha informada não corresponde ao hash BCrypt armazenado na tabela users.

25. O sistema compara a senha fornecida com o campo password_hash via BCrypt.
26. A comparação falha.
27. O sistema exibe MSG003: 'E-mail ou senha incorretos.' – sem especificar qual campo está errado (RN-03).
28. Nenhum token é gerado. O usuário permanece na tela de login.

(E02) – Senha fraca no cadastro

Pré-condições: A senha informada não atende aos critérios mínimos de segurança (RN-01).

29. O sistema valida a senha em tempo real (onChange) e ao submeter o formulário.
30. A senha não atende a: mínimo 8 caracteres, uma maiúscula e um número.
31. O sistema exibe MSG002: 'Senha fraca. Use ao menos 8 caracteres, uma letra maiúscula e um número.'
32. O botão 'Criar Conta' permanece desabilitado. Nenhum registro é inserido.

(E03) – Refresh token expirado ou inválido

Pré-condições: O refresh token enviado não existe no Redis 7 ou está expirado.

33. O sistema busca o refresh token no Redis 7 e não o encontra ou ele está expirado.
34. O sistema retorna HTTP 401 Unauthorized.
35. O cliente redireciona o usuário para a tela de login (/login).

4. Pós-condições

- Sucesso no cadastro: registro criado na tabela users com id (UUID), email, password_hash (BCrypt), created_at e updated_at. Usuário autenticado e redirecionado ao /dashboard.
- Sucesso no login: access token JWT (15min) e refresh token (7 dias) gerados. Usuário redirecionado ao /dashboard.
- Falha: nenhum registro alterado na tabela users. Mensagem de erro exibida ao usuário.

5. Requisitos Especiais

(RE01) – Desempenho de Autenticação

O endpoint POST /auth/login deve responder em menos de 300ms no percentil 95 (P95) com até 500 usuários simultâneos – RNF-01, Seção 6 Doc. de Visão Equipe 9.

(RE02) – Segurança de Armazenamento

A senha nunca deve ser armazenada em texto puro. O campo password_hash usa BCrypt com fator de custo 12 (~300–400ms por hash) – RNF-03, Seção 6 Doc. de Visão.

(RE03) – Rate Limiting

O sistema limita tentativas de login a 60 requisições por minuto por IP para prevenir ataques de força bruta – RNF-03, Seção 6 Doc. de Visão Equipe 9.

6. Regras de Negócio**(RN01) – Critérios de Senha Forte**

A senha deve conter no mínimo 8 caracteres, ao menos uma letra maiúscula e um número. Senhas que não atendam exibem MSG002 e bloqueiam o cadastro.

(RN02) – Geração de Tokens JWT

Após autenticação bem-sucedida, o sistema gera: (1) access token JWT com validade de 15 minutos; (2) refresh token com validade de 7 dias, armazenado no Redis 7 e enviado como cookie HttpOnly se 'Manter conectado' estiver marcado.

(RN03) – Mensagem de Erro Genérica no Login

Em caso de credenciais inválidas, o sistema exibe MSG003 sem especificar qual campo (e-mail ou senha) está incorreto, prevenindo enumeração de usuários (OWASP).

(RN04) – Unicidade de E-mail

O campo email na tabela users possui constraint UNIQUE. Tentativa de cadastro com e-mail já existente resulta em MSG001 e não insere registro no banco.

7. Interfaces com Usuários**(I01) – Tela de Cadastro (/cadastro)**

Layout centralizado com logo, título 'Criar conta', formulário e link 'Já tenho conta'. Responsivo para web e mobile (iOS 14+ / Android 10+). Protótipo em Figma – Sprint 1.

Campos:

No.	Nome	Descrição	Valores Válidos	Tipo	Restrições
1	E-mail	Endereço de e-mail do usuário	Formato válido RFC 5322	Alfanumérico	Obrigatório; VARCHAR 255; UNIQUE na tabela users
2	Senha	Senha de acesso	≥8 chars, 1 maiúscula, 1 número	Alfanumérico	Obrigatório; exibir barra de força; armazenado como BCrypt
3	Confirmar Senha	Confirmação da senha	Igual ao campo Senha	Alfanumérico	Obrigatório; validado em tempo real

Comandos:

No.	Nome	Ação	Restrições
1	Criar Conta	Valida campos e insere registro na tabela users via POST /auth/register.	Habilitado somente se todos os campos são válidos
2	Entrar (link)	Redireciona para a tela de login (/login).	Sempre habilitado

(I02) – Tela de Login (/login)

Layout centralizado com logo, título 'Entrar', formulário, checkbox e link 'Criar conta'.
Responsivo para web e mobile. Protótipo em Figma – Sprint 1.

Campos:

No.	Nome	Descrição	Valores Válidos	Tipo	Restrições
1	E-mail	E-mail cadastrado na tabela users	Formato válido RFC 5322	Alfanumérico	Obrigatório
2	Senha	Senha do usuário	Qualquer valor	Alfanumérico	Obrigatório; campo tipo password
3	Manter conectado	Gera refresh token de 7 dias em cookie HttpOnly	Marcado/Desmarcado	Booleano	Opcional; default: desmarcado

Comandos:

No.	Nome	Ação	Restrições
1	Entrar	Autentica via POST /auth/login e redireciona ao /dashboard.	Habilitado somente se e-mail e senha preenchidos
2	Criar conta (link)	Redireciona para a tela de cadastro (/cadastro).	Sempre habilitado
3	Esqueci minha senha (link)	Redireciona para fluxo de recuperação de senha (Fase 2).	Sempre habilitado

8. Demais Interfaces**8.1 Interfaces de Software**

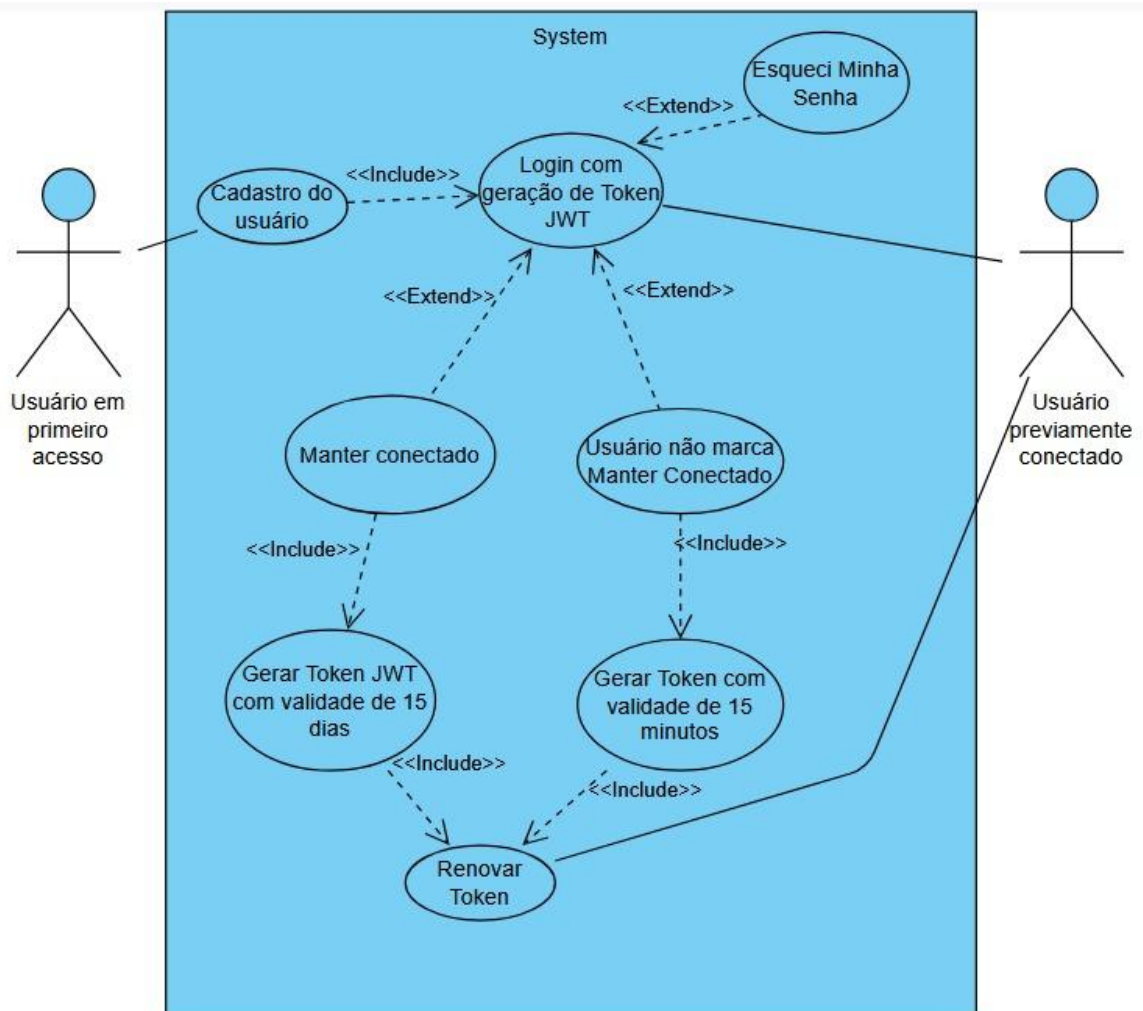
No.	Nome	Ator	Descrição
1	API REST – Spring Boot 3	Sistema	Endpoint POST /auth/register e POST /auth/login. Retorna tokens JWT. Backend Java 21 + Spring Security.

2	Redis 7	Sistema	Armazenamento e validação de refresh tokens com TTL de 7 dias.
3	PostgreSQL 16	Sistema	Persistência da tabela users (id, email, password_hash, created_at, updated_at).

8.2 Interfaces de Hardware

No.	Nome	Hardware	Descrição
1	Dispositivo Web	PC/Notebook	Navegador Chrome 110+, Firefox 110+ ou Safari 16+ – Seção 4.4 Doc. de Visão.
2	Dispositivo Mobile	Smartphone/Tablet	iOS 14+ ou Android 10+ via React Native (Expo) – Seção 4.4 Doc. de Visão.

9. Diagrama de Caso de Uso



10. Observações

- O fluxo de recuperação de senha ('Esqueci minha senha') está fora do escopo da Fase 1 (NR-06) e será implementado na Fase 2.
- A validação de e-mail ocorre em tempo real (evento onBlur) antes da submissão do formulário (RAP001 – HU-01).
- Referência: HU-01 (Equipe 9, v1.01, Mar/2026); RF0001; Seção 9.2 do Doc. de Visão (tabela users).

UC-02 – Criar e Gerenciar Ficha de Personagem D&D 5e

ID do Caso de Uso: UC-02

Nome: Criar e Gerenciar Ficha de Personagem D&D 5e

Atores: Jogador – usuário autenticado (JWT válido)

Prioridade: Crítico – Prioridades 2 e 3, Fase 1 MVP (Seção 5 Doc. de Visão Equipe 9)

Referência: HU-02; RF0002; Seção 3.2 (Necessidades 1 e 2); Seção 9.2 (tabela characters); SRD D&D 5e

1. Descrição

Este caso de uso permite que o jogador autenticado crie, visualize, edite e exclua fichas de personagens D&D 5e. O sistema calcula automaticamente os modificadores de atributo ($\text{floor}((\text{valor}-10)/2)$), o Bônus de Proficiência por nível e a Classe de Armadura. Os dados são persistidos na tabela characters do PostgreSQL 16, com atributos armazenados em JSONB para suportar futuros sistemas (ex: Ordem Paranormal – Fase 3).

2. Pré-condições

- O usuário está autenticado (possui access token JWT válido – UC-01 concluído).
- O sistema está disponível e o banco de dados PostgreSQL 16 está acessível.
- O compêndio SRD D&D 5e (classes e raças) está carregado no banco de dados.

3. Fluxo de Eventos

3.1 Fluxo Básico

36. O jogador acessa 'Meus Personagens' e clica em 'Novo Personagem' (/personagens/novo).
37. O sistema exibe o formulário de criação da Ficha com as seções: Identidade (Nome, Nível, Raça, Classe, Subclasse, Idade, Aparência) e Narrativa (História do personagem). Os dados de combate (HP, CA, atributos) e o inventário são gerenciados na tela Inventário, acessada pelo botão correspondente dentro da Ficha.
38. O jogador preenche na tela Ficha: Nome (VARCHAR 100), Classe (combo – campo class VARCHAR 50), Subclasse (combo – campo subclass VARCHAR 50), Raça (combo – campo race VARCHAR 50), Nível (INT 1–20), Idade, História do personagem (textarea) e Aparência (upload de imagem) [RN-01].

39. Os seis atributos base (Força/Vermelho, Destreza/Verde, Constituição/Prata, Inteligência/Azul, Sabedoria/Roxo e Carisma/Âmbar – INT 1–20 cada) são preenchidos e gerenciados na tela Inventário [RN-02].
40. A cada atributo digitado, o sistema calcula e exibe imediatamente o modificador correspondente: $\text{floor}((\text{valor}-10)/2)$ [RN-02].
41. O sistema calcula e exibe automaticamente o Bônus de Proficiência com base no Nível informado [RN-03].
42. O jogador clica em 'Salvar Personagem'.
43. O sistema valida todos os campos e envia requisição POST /characters com o token JWT.
44. O sistema persiste a ficha na tabela characters, vinculando ao user_id (FK UUID) do jogador autenticado, e armazena os atributos como JSONB.
45. O sistema exibe MSG005: 'Personagem salvo com sucesso!' e redireciona para a visualização da ficha.

3.2 Subfluxos

(S01) – Edição de Ficha Existente

46. O jogador acessa 'Meus Personagens' e clica no personagem que deseja editar.
47. O sistema carrega os dados da ficha do banco de dados via GET /characters/{id}.
48. O sistema exibe o formulário pré-preenchido com os dados atuais.
49. O jogador altera os campos desejados.
50. Os cálculos automáticos são atualizados em tempo real conforme os campos são alterados.
51. O jogador clica em 'Salvar Alterações'.
52. O sistema envia requisição PUT /characters/{id} e atualiza o registro na tabela characters.
53. O sistema exibe MSG005 e retorna à visualização da ficha atualizada.

(S02) – Exclusão de Ficha

54. O jogador acessa a ficha que deseja excluir e clica em 'Excluir Personagem'.
55. O sistema exibe diálogo de confirmação: 'Tem certeza? Esta ação não pode ser desfeita.'
56. O jogador confirma a exclusão.
57. O sistema envia requisição DELETE /characters/{id} e remove o registro da tabela characters (hard delete).
58. O sistema redireciona para 'Meus Personagens' e exibe MSG: 'Personagem excluído.'

3.3 Fluxos Alternativos

(A01) – Jogador não preenche todos os campos obrigatórios

Pré-condições: Um ou mais campos obrigatórios (Nome, Classe, Raça, Nível ou atributos) estão vazios ao clicar em 'Salvar'.

- 59. O sistema identifica os campos obrigatórios não preenchidos.
- 60. O sistema destaca os campos com borda vermelha e exibe mensagens de erro abaixo de cada um (RAP003).
- 61. O botão 'Salvar Personagem' permanece desabilitado.
- 62. Nenhum registro é inserido na tabela characters.

Retorno: O jogador permanece no formulário para corrigir os campos.

(A02) – Jogador consulta lista de personagens

Pré-condições: O jogador está autenticado e acessa 'Meus Personagens'.

- 63. O sistema realiza GET /characters listando todas as fichas do usuário autenticado (WHERE user_id = JWT.sub).
- 64. O sistema exibe a lista com nome, classe, raça e nível de cada personagem.
- 65. O jogador seleciona um personagem para visualizar ou editar.

Retorno: Caso o jogador selecione editar, inicia-se o Subfluxo S01.

3.4 Exceções

(E01) – Nível fora do intervalo válido (1–20)

Pré-condições: O valor informado no campo Nível é menor que 1 ou maior que 20.

- 66. O sistema valida o campo Nível em tempo real (onChange) e ao submeter.
- 67. O valor está fora do intervalo 1–20 (campo level INT na tabela characters).
- 68. O sistema exibe MSG004: 'Nível inválido. Informe um valor entre 1 e 20.'
- 69. O campo Nível recebe destaque vermelho. O botão Salvar é desabilitado.
- 70. Nenhum registro é inserido na tabela characters.

(E02) – Token JWT expirado durante o preenchimento

Pré-condições: O access token JWT do jogador expirou (15 minutos) enquanto preenchia o formulário.

- 71. O jogador clica em 'Salvar Personagem'.
- 72. O sistema tenta o refresh do token via POST /auth/refresh.
- 73. Se bem-sucedido: a ficha é salva normalmente (retorna ao passo 8 do fluxo básico).
- 74. Se refresh falhar: o sistema exibe MSG: 'Sua sessão expirou. Faça login novamente.' e redireciona para /login. Os dados do formulário são preservados no localStorage temporariamente.

(E03) – Atributo com valor inválido (fora de 1–20)

Pré-condições: Um ou mais atributos possuem valor fora do intervalo 1–20.

- 75. O sistema valida cada atributo em tempo real (onChange).

- 76. O atributo inválido recebe borda vermelha com mensagem: 'Valor deve ser entre 1 e 20.'
- 77. O botão Salvar permanece desabilitado.
- 78. Nenhum registro é inserido na tabela characters.

4. Pós-condições

- Sucesso na criação: registro inserido na tabela characters com id (UUID), user_id (UUID FK), name, class, subclass, race, level, age, history, appearance_url, attributes (JSONB com FOR/DES/CON/INT/SAB/CAR com cores associadas), hp, max_hp e ac. Usuário vê a ficha criada.
- Sucesso na edição: campos atualizados na tabela characters. Valores calculados (modificadores, bônus) recalculados.
- Sucesso na exclusão: registro removido da tabela characters. Lista de personagens atualizada.
- Falha: nenhum registro alterado no banco. Mensagem de erro exibida. Usuário permanece no formulário.

5. Requisitos Especiais

(RE01) – Cálculo em Tempo Real

Os modificadores e o Bônus de Proficiência devem ser calculados e exibidos imediatamente a cada alteração de campo (evento onChange), sem necessidade de salvar – RAP001 e RAP002 da HU-02.

(RE02) – Persistência em JSONB

Os atributos (FOR/Vermelho, DES/Verde, CON/Prata, INT/Azul, SAB/Roxo, CAR/Âmbar) são armazenados em JSONB no campo attributes da tabela characters para suportar futuros sistemas de RPG como Ordem Paranormal (Fase 3) – Seção 9.2 Doc. de Visão. Os campos de roleplay (subclasse, história, idade, aparência) são armazenados na mesma tabela; HP, CA e inventário são gerenciados na tela Inventário.

(RE03) – Desempenho

O endpoint POST /characters deve responder em menos de 300ms (P95) – RNF-01, Seção 6 Doc. de Visão Equipe 9.

6. Regras de Negócio

(RN01) – Intervalo Válido do Nível

O campo Nível (level INT) aceita somente valores inteiros de 1 a 20. Valores fora deste intervalo disparam MSG004 e bloqueiam o salvamento.

(RN02) – Cálculo de Modificador de Atributo

Modificador = $\text{floor}((\text{valor} - 10) / 2)$. Exemplos: valor 1 $\rightarrow$ -5; valor 10 $\rightarrow$ 0; valor 11 $\rightarrow$ 0; valor 16 $\rightarrow$ +3; valor 20 $\rightarrow$ +5. Fórmula conforme SRD D&D 5e (Seção 1.4.1 Doc. de Visão).

(RN03) – Cálculo de Bônus de Proficiência

Baseado no nível conforme tabela oficial D&D 5e: níveis 1–4 = +2; 5–8 = +3; 9–12 = +4; 13–16 = +5; 17–20 = +6. Campo somente leitura – calculado pelo sistema.

(RN04) – Vínculo com Usuário

Toda ficha criada deve estar vinculada ao user_id (UUID FK) do usuário autenticado (extraído do token JWT). Fichas de outros usuários não são acessíveis (HTTP 403 Forbidden).

7. Interfaces com Usuários

(I01) – Tela de Criação/Edição de Ficha (/personagens/novo)

Formulário dividido em duas seções: Identidade (Nome, Nível, Raça, Classe, Subclasse, Idade, Aparência) e Narrativa (História do personagem). Um botão Inventário dá acesso à tela de Inventário onde ficam HP, CA, atributos e itens. Protótipo em Figma – Sprint 1.

Campos:

No.	Nome	Descrição	Valores Válidos	Tipo	Restrições
1	Nome	Nome fictício do personagem	Texto livre	Alfanumérico	Obrigatório; VARCHAR 100; campo name na tabela characters
2	Classe	Classe D&D 5e (combo do SRD)	Lista do SRD 5e	Combo	Obrigatório; VARCHAR 50; campo class
3	Raça	Raça do personagem (combo do SRD)	Lista do SRD 5e	Combo	Obrigatório; VARCHAR 50; campo race
4	Nível	Nível atual (1–20)	1 a 20	Inteiro	Obrigatório; campo level INT
5	Força / Modificador	Atributo base + modificador calculado	1 a 20 / -5 a +5	Inteiro	Obrigatório; parte do JSONB attributes
6	Destreza / Modificador	Atributo base + modificador calculado	1 a 20 / -5 a +5	Inteiro	Obrigatório; parte do JSONB attributes
7	Constituição / Modificador	Atributo base + modificador calculado	1 a 20 / -5 a +5	Inteiro	Obrigatório; parte do JSONB attributes

8	Inteligência / Modificador	Atributo base + modificador calculado	1 a 20 / –5 a +5	Inteiro	Obrigatório; parte do JSONB attributes
9	Sabedoria / Modificador	Atributo base + modificador calculado	1 a 20 / –5 a +5	Inteiro	Obrigatório; parte do JSONB attributes
10	Carisma / Modificador	Atributo base + modificador calculado	1 a 20 / –5 a +5	Inteiro	Obrigatório; parte do JSONB attributes
11	Bônus de Proficiência	Calculado pelo sistema com base no Nível	+2 a +6	Inteiro	Somente leitura; campo calculado – RN-03
12	HP Máximo	Pontos de vida máximos – campo max_hp INT	Calculado	Inteiro	Somente leitura; campo max_hp na tabela characters; exibido e editado na tela Inventário
13	CA	Classe de Armadura – campo ac INT	Calculado	Inteiro	Somente leitura; campo ac na tabela characters; exibido e editado na tela Inventário

Comandos:

No.	Nome	Ação	Restrições
1	Salvar Personagem	Envia POST /characters (criação) ou PUT /characters/{id} (edição). Persiste na tabela characters.	Habilitado somente se todos os campos obrigatórios são válidos
2	Cancelar	Descarta alterações e redireciona para 'Meus Personagens'.	Sempre habilitado
3	Excluir Personagem	Exibe diálogo de confirmação antes de DELETE /characters/{id}.	Habilitado somente na edição de ficha existente

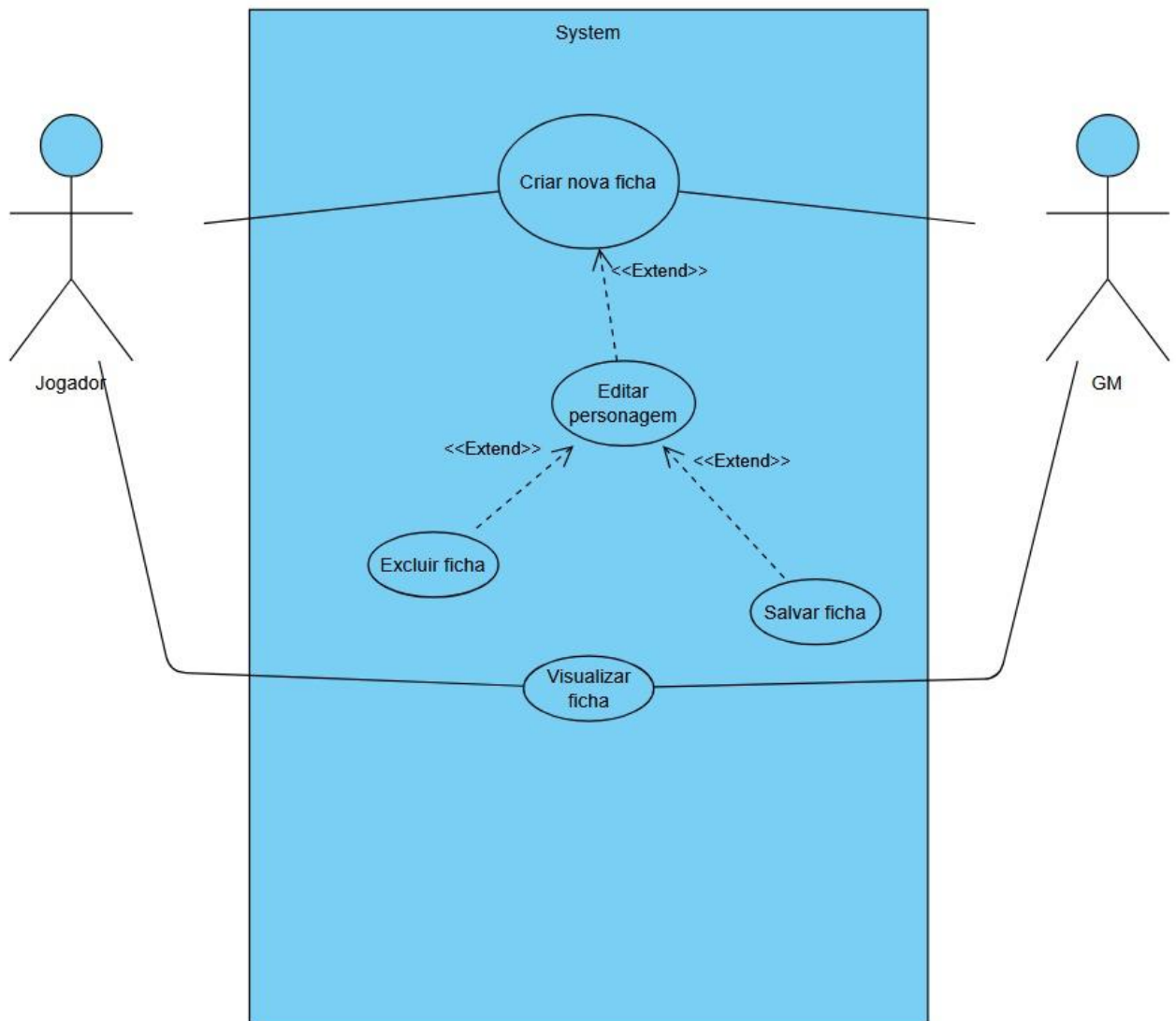
8. Demais Interfaces**8.1 Interfaces de Software**

No.	Nome	Ator	Descrição
1	API REST – /characters	Jogador	POST (criar), GET (listar/visualizar), PUT (editar), DELETE (excluir). Spring Boot 3 + JPA + PostgreSQL 16.
2	Compêndio SRD D&D 5e	Sistema	Fornecer listas de classes e raças para os combos do formulário. Dados carregados do banco de dados.
3	PostgreSQL 16	Sistema	Persistência da tabela characters com todos os campos conforme Seção 9.2 do Doc. de Visão.

8.2 Interfaces de Hardware

No.	Nome	Hardware	Descrição
1	Dispositivo Web	PC/Notebook	Navegador Chrome 110+, Firefox 110+ ou Safari 16+.
2	Dispositivo Mobile	Smartphone/Tablet	iOS 14+ ou Android 10+ via React Native (Expo).

9. Diagrama de Caso de Uso



10. Observações

- O cálculo de HP máximo completo (dado de vida da classe × nível + bônus CON) será detalhado na especificação da engine de regras D&D 5e (Arquitetura Técnica – Seção 2.4).
- Atributos armazenados em JSONB (campo attributes) garantem extensibilidade para Ordem Paranormal (Fase 3) sem alteração do esquema – Seção 9.2 Doc. de Visão Equipe 9.
- Referência: HU-02 (Equipe 9, v1.01, Mar/2026); RF0002; SRD D&D 5e (Seção 1.4.1 Doc. de Visão).

UC-03 – Rolar Dados Virtuais

ID do Caso de Uso: UC-03

Nome: Rolar Dados Virtuais

Atores: Jogador, Mestre (GM) – usuários autenticados (JWT válido)

Prioridade: Crítico – Prioridade 4, Fase 1 MVP (Seção 5 Doc. de Visão Equipe 9)

Referência: HU-03; RF0003; Seção 3.2 (Necessidade 2); Seção 8.2 (WebSocket); Seção 9.2 (tabela dice_rolls)

1. Descrição

Este caso de uso permite que jogadores e mestres realizem rolagens de dados virtuais utilizando fórmulas livres compostas (ex: $2d6 + 3D10 + 2$) ou atalhos rápidos (d4–d100). O processamento ocorre no backend (java.security.SecureRandom) para garantir integridade. Cada rolagem é registrada na tabela dice_rolls e, em sessões ativas, transmitida em tempo real via WebSocket (STOMP/SockJS) para todos os participantes da sala – Seção 8.2 do Doc. de Visão Equipe 9.

2. Pré-condições

- O usuário está autenticado (possui access token JWT válido – UC-01 concluído).
- O sistema está disponível e o banco de dados PostgreSQL 16 está acessível.
- Para transmissão em sessão (CA5): existe sessão ativa com code válido e o usuário está conectado via WebSocket.

3. Fluxo de Eventos

3.1 Fluxo Básico

79. O usuário acessa a área de rolagem (/dados) ou a sala de sessão (/sessoes/{codigo}).
80. O sistema exibe o campo de fórmula, os atalhos rápidos (d4, d6, d8, d10, d12, d20, d100) e o histórico das últimas 50 rolagens.
81. O usuário digita uma fórmula no campo (ex: '2d6+3') ou clica em um atalho rápido.
82. O sistema valida a fórmula em tempo real (formato de fórmula livre com múltiplos grupos NdX separados por + ou –, mais modificador opcional) [RN-01].
83. O usuário clica em 'Rolar' (ou o atalho executa imediatamente).
84. O sistema envia requisição POST /dice/roll com a fórmula e o token JWT.
85. O backend processa a rolagem usando java.security.SecureRandom, gerando resultados individuais para cada dado.

- 86. O sistema insere registro na tabela dice_rolls: id (UUID), user_id (UUID FK), session_id (UUID – NULL se fora de sessão), formula (VARCHAR 50), individual_results (JSONB), total (INT), rolled_at (TIMESTAMP) [RN-02].
- 87. O sistema retorna e exibe o resultado: fórmula, resultados individuais e total. Ex: '2d6+3 → [4, 2] + 3 = 9'.
- 88. O histórico das últimas 50 rolagens é atualizado automaticamente.
- 89. Se em sessão ativa: o resultado é transmitido via WebSocket ao tópico /topic/session.{codigo} com o nome do usuário que rolou [RN-03].

3.2 Subfluxos

(S01) – Rolagem via Atalho Rápido

- 90. O usuário clica em um dos botões de atalho: d4, d6, d8, d10, d12, d20 ou d100.
- 91. O sistema define automaticamente a fórmula como '1dX' (ex: clicar d20 → fórmula '1d20').
- 92. O sistema executa imediatamente o passo 6 do fluxo básico sem necessidade de digitar a fórmula.
- 93. O fluxo básico continua a partir do passo 7.

(S02) – Visualização do Histórico

- 94. O usuário acessa a seção de histórico na área de rolagem.
- 95. O sistema consulta a tabela dice_rolls via GET /dice/history (WHERE user_id = JWT.sub ORDER BY rolled_at DESC LIMIT 50).
- 96. O sistema exibe as últimas 50 rolagens com: fórmula, resultados individuais, total e data/hora.

3.3 Fluxos Alternativos

(A01) – Resultado máximo – Acerto Crítico (d20 = 20)

Pré-condições: O resultado de uma rolagem com d20 é igual a 20.

- 97. O sistema detecta resultado 20 em um dado d20.
- 98. O sistema exibe destaque especial: borda dourada e texto 'Crítico!' na área de resultado (RAP002 – HU-03).
- 99. A rolagem é registrada normalmente na tabela dice_rolls.

Retorno: O fluxo básico continua a partir do passo 10.

(A02) – Rolagem fora de sessão ativa

Pré-condições: O usuário está na área /dados (sem sala de sessão ativa).

- 100. O usuário realiza a rolagem normalmente.
- 101. O campo session_id na tabela dice_rolls é inserido como NULL [RN-02].
- 102. Nenhuma transmissão via WebSocket ocorre.

103. O resultado é exibido apenas para o usuário que rolou.
Retorno: O fluxo básico retorna ao passo 10.

3.4 Exceções

(E01) – Fórmula inválida

Pré-condições: A fórmula digitada não contém nenhum grupo válido no formato NdX (ex: 'abc', '0d6', '2d', texto sem dados).

- 104. O sistema valida a fórmula em tempo real (onChange) e ao clicar em 'Rolar'.
- 105. A fórmula não corresponde ao formato válido.
- 106. O sistema exibe MSG006: 'Fórmula inválida. Use dados no formato NdX separados por + ou – (ex: 2d6 + 3D10 + 2).'
- 107. O campo recebe borda vermelha. O botão 'Rolar' permanece desabilitado.
- 108. Nenhum registro é inserido na tabela dice_rolls.

(E02) – Token JWT expirado durante a rolagem

Pré-condições: O access token JWT expirou (15 minutos) ao clicar em 'Rolar'.

- 109. O sistema tenta renovar via POST /auth/refresh com o refresh token.
- 110. Se bem-sucedido: a rolagem é executada normalmente.
- 111. Se falhar: o sistema exibe MSG: 'Sessão expirada. Faça login novamente.' e redireciona para /login.

(E03) – Desconexão WebSocket durante sessão

Pré-condições: A conexão WebSocket do usuário é interrompida durante uma sessão ativa.

- 112. O cliente detecta a perda de conexão WebSocket.
- 113. O cliente tenta reconectar automaticamente (3 tentativas com backoff exponencial).
- 114. Se reconectado: o usuário retorna ao estado da sala (estado recuperado do Redis 7).
- 115. Se falhar: o sistema exibe MSG: 'Conexão perdida. Tente reconectar.' com botão de ação.

4. Pós-condições

- Sucesso: registro inserido na tabela dice_rolls com id, user_id, session_id (NULL ou ID da sessão), formula, individual_results (JSONB), total e rolled_at. Resultado exibido ao usuário.
- Em sessão ativa: resultado transmitido via WebSocket a todos os participantes da sala.
- Falha por fórmula inválida: nenhum registro no banco, MSG006 exibida ao usuário.

- Falha por token: usuário redirecionado para /login. Nenhum registro no banco.

5. Requisitos Especiais

(RE01) – Aleatoriedade Segura

A rolagem usa `java.security.SecureRandom` no backend para garantir imprevisibilidade criptográfica. O processamento ocorre no servidor – nunca no cliente – para evitar manipulação de resultados.

(RE02) – Latência em Sessão Online

A transmissão via WebSocket (STOMP/SockJS) deve chegar a todos os participantes da sala em menos de 1 segundo em condições normais de rede (4G com latência até 150ms – Seção 7.2 Doc. de Visão Equipe 9).

(RE03) – Desempenho

O endpoint `POST /dice/roll` deve responder em menos de 300ms (P95) com 500 usuários simultâneos – RNF-01, Seção 6 Doc. de Visão.

6. Regras de Negócio

(RN01) – Formato Válido de Fórmula

Fórmulas aceitas: expressões compostas por um ou mais grupos `NdX` (N = número de dados 1–99; X = tipo de dado 4, 6, 8, 10, 12, 20 ou 100), separados por + ou –, com modificador inteiro opcional ao final. Letras maiúsculas ou minúsculas são aceitas no “d”. Exemplos válidos: `1d20`, `2d6+3`, `2d6 + 3D10 + 2`, `4d4-1`. O campo `formula` (VARCHAR 50) deve comportar a expressão completa. Qualquer expressão sem ao menos um grupo `NdX` válido é inválida e dispara MSG006.

(RN02) – `session_id` Nullable

O campo `session_id` na tabela `dice_rolls` é nullable. Rolagens realizadas fora de sessão ativa são inseridas com `session_id = NULL`. Nenhuma transmissão WebSocket ocorre neste caso – Seção 9.2 Doc. de Visão Equipe 9.

(RN03) – Transmissão via WebSocket em Sessão

Em sessão ativa, o resultado é publicado no tópico `/topic/session.{codigo}` via STOMP/SockJS. Todos os participantes conectados recebem: nome do jogador, fórmula, resultados individuais e total – Seção 8.2 Doc. de Visão.

(RN04) – Limite de Histórico

O histórico exibe apenas as últimas 50 rolagens do usuário (ORDER BY `rolled_at` DESC LIMIT 50 na tabela `dice_rolls`). Rolagens mais antigas permanecem no banco mas não são exibidas na interface.

7. Interfaces com Usuários

(I01) – Área de Rolagem de Dados (/dados)

Layout dividido em: (1) Campo de fórmula + botão Rolar; (2) Atalhos rápidos d4/d6/d8/d10/d12/d20/d100; (3) Área de resultado com animação; (4) Histórico lateral das últimas 50 rolagens. Responsivo. Protótipo em Figma – Sprint 1.

Campos:

No.	Nome	Descrição	Valores Válidos	Tipo	Restrições
1	Fórmula	Fórmula da rolagem (expressão livre com grupos NdX)	Ex: 2d6+3, 1d20	Alfanumérico	Obrigatório; VARCHAR 50; campo formula na tabela dice_rolls
2	Resultado	Exibe fórmula, individuais e total após rolar	Calculado	Texto	Somente leitura; gerado pelo backend com SecureRandom
3	Histórico	Lista das 50 últimas rolagens do usuário	N/A	Lista	Somente leitura; ORDER BY rolled_at DESC LIMIT 50

Comandos:

No.	Nome	Ação	Restrições
1	Rolar	Envia POST /dice/roll com a fórmula. Exibe resultado e atualiza histórico.	Habilitado somente com fórmula válida
2	d4 / d6 / d8 / d10 / d12 / d20 / d100	Executa rolagem '1dX' imediatamente via POST /dice/roll.	Sempre habilitado (usuário autenticado)

(I02) – Feed de Rolagens em Sessão Online (/sessoes/{codigo})

Feed em tempo real exibido lateralmente na sala de sessão. Rolagens do usuário têm destaque diferente das dos outros participantes. Transmitido via WebSocket (STOMP). Protótipo em Figma – Sprint 1.

Campos:

No.	Nome	Descrição	Valores Válidos	Tipo	Restrições
1	Feed de Rolagens	Lista em tempo real das rolagens de todos os participantes	N/A	Lista	Somente leitura; atualizado via WebSocket tópico /topic/session.{codigo}
2	Nome do Jogador	Identifica quem realizou a rolagem	Nome do usuário	Texto	Somente leitura; extraído do token JWT

Comandos:

No.	Nome	Ação	Restrições
1	Rolar (na sessão)	Rola e transmite via WebSocket para todos os participantes.	Habilitado com fórmula válida e sessão ativa

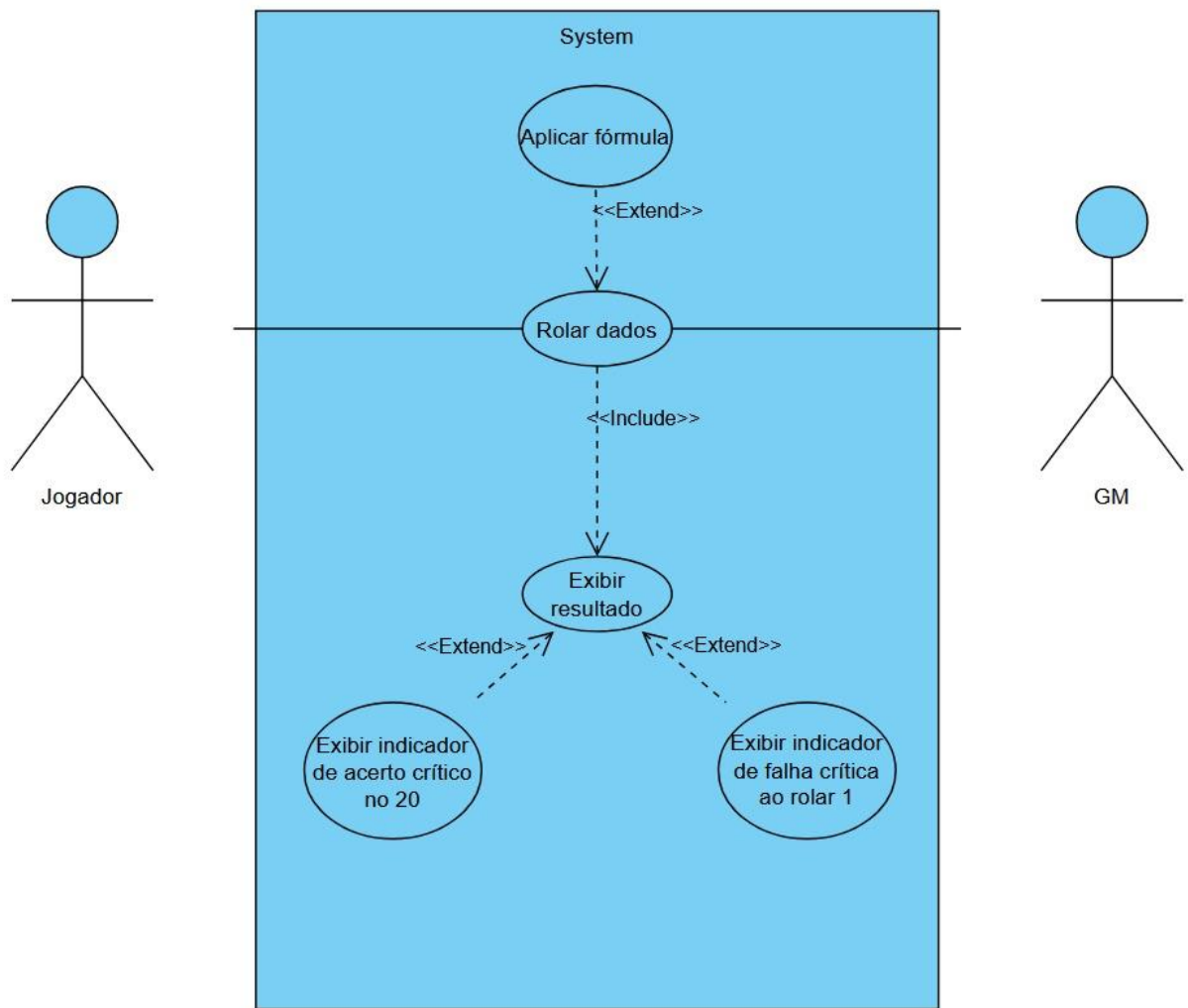
8. Demais Interfaces**8.1 Interfaces de Software**

No.	Nome	Ator	Descrição
1	API REST – /dice/roll	Jogador/Mestre	POST /dice/roll: processa rolagem com SecureRandom, persiste em dice_rolls e retorna resultado.
2	WebSocket (STOMP/SockJS)	Sistema	Publica resultado no tópico /topic/session.{codigo} para transmissão em tempo real – Seção 8.2 Doc. de Visão.
3	PostgreSQL 16	Sistema	Persistência da tabela dice_rolls (todos os campos conforme Seção 9.2 Doc. de Visão).
4	Redis 7	Sistema	Estado das salas de sessão ao vivo. Armazena participantes conectados e últimas rolagens.

8.2 Interfaces de Hardware

No.	Nome	Hardware	Descrição
1	Dispositivo Web	PC/Notebook	Navegador Chrome 110+, Firefox 110+ ou Safari 16+.
2	Dispositivo Mobile	Smartphone/Tablet	iOS 14+ ou Android 10+ via React Native (Expo).

9. Diagrama de Caso de Uso



10. Observações

- A funcionalidade de sessões online completa (criação de sala, gerenciamento de GM, código de acesso) é escopo da Fase 2 (NR-03). O CA5 da HU-03 cobre apenas a transmissão via WebSocket em sessão existente.
- O histórico completo de sessões encerradas será implementado na Fase 2 (Prioridade 9 – Seção 5 Doc. de Visão Equipe 9).
- Referência: HU-03 (Equipe 9, v1.01, Mar/2026); RF0003; Seção 8.2 e 9.2 do Doc. de Visão Equipe 9.

8. Rastreabilidade entre os Requisitos

Requisito	Fonte (HU)	Dific.	Rastreabilidade (Doc. de Visão)	Observações
RF0001	HU-01	Alta	Necessidade 3 (Crítico)	RF0002, RF0003 (dependem de autenticação)
RF0001.1	HU-01 CA1–CA3	Alta	Seção 9.2 (tabela users)	RF0001.2, RF0001.3
RF0001.2	HU-01 CA5–CA6	Alta	Seção 6 (RNF Segurança)	RF0001.3, RF0002, RF0003
RF0002	HU-02	Alta	Necessidades 1 e 2 (Crítico)	RF0001 (pré-requisito)
RF0002.2	HU-02 CA2	Alta	Seção 4.1 (Funcionalidade 2)	RF0002.3
RF0002.3	HU-02 CA4	Alta	SRD D&D 5e (Seção 1.4.1)	RF0002.1
RF0003	HU-03	Alta	Necessidade 2 (Crítico)	RF0001 (pré-requisito)
RF0003.1	HU-03 CA1–CA2	Alta	Seção 9.2 (tabela dice_rolls)	RF0003.3, RF0003.4
RF0003.4	HU-03 CA5	Média	Seção 8.2 (WebSocket STOMP)	RF0003.1, RF0003.3

9. Não Requisitos (Fora do Escopo da Fase 1)

ID	Descrição	Responsável	Observação
NR-01	Suporte ao sistema Ordem Paranormal	Lucas Guerra	Fase 3 do Roadmap
NR-02	Parser automático de PDF de regras	Luiz Philipe	Fase 3 do Roadmap
NR-03	Sessões online com WebSocket em produção	Luis Nunes	Fase 2 do Roadmap
NR-04	Aplicativo mobile iOS e Android publicado nas lojas	João Pedro Nunes	Fase 2 / Fase 4
NR-05	Compêndio completo de magias, equipamentos e classes D&D 5e	João Pedro Nunes e Luis Nunes	Fase 2
NR-06	Testes de acessibilidade (WCAG) e usabilidade aprofundada	Leonardo e Luiz Philipe	Fase 2+

10. Riscos Identificados

ID	Descrição	Contingência	Requisitos Afetados
R-01	Ambiente Docker/PostgreSQL/Redis instável	Usar Testcontainers em testes de integração; script de reset rápido.	RF0001, RF0002, RF0003
R-02	Cobertura de testes abaixo de 80%	Integrar JaCoCo ao GitHub Actions e bloquear merge se regredir.	Todos
R-03	Mudança de requisito durante os testes	Congelar escopo das HUs 01–03 antes do início dos testes da Fase 1.	RF0001, RF0002, RF0003
R-04	SRD D&D 5e indisponível para carga do compêndio	Manter dataset JSON local com subset básico (classes e raças essenciais).	RF0002
R-05	Instabilidade do WebSocket em ambiente de testes	Criar testes de integração com dois clientes STOMP simulados.	RF0003.4
R-06	Equipe reduzida – bottleneck QA e dev simultâneos	Automação desde Sprint S1; CI/CD com GitHub Actions para execução contínua.	Todos